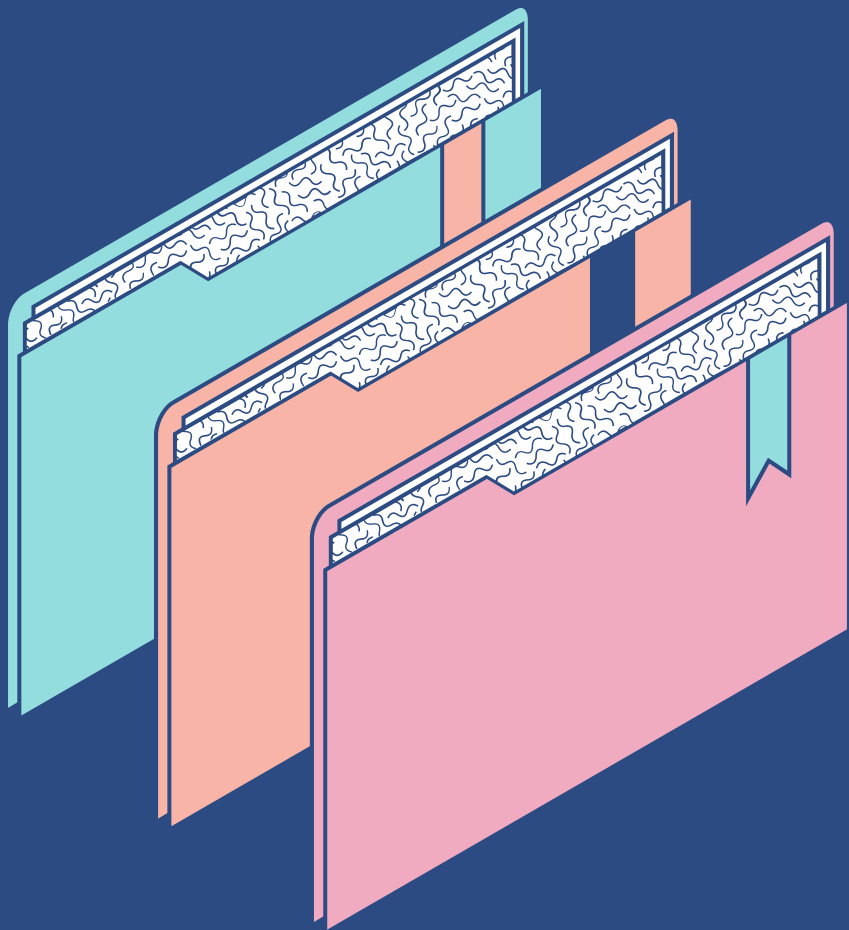




КАФЕДРА №13

Назначение и область применения исключений

ЛИТВИНОВ ВЛАДИСЛАВ

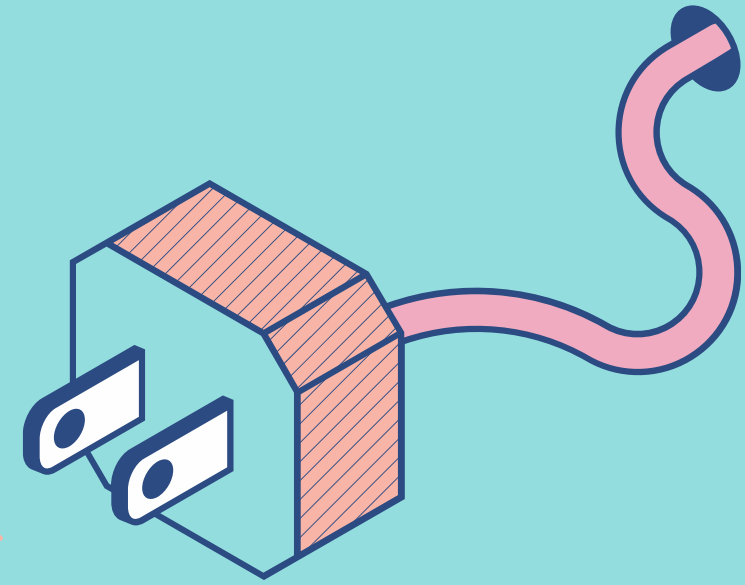
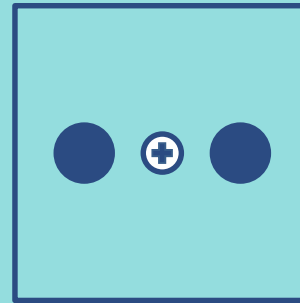
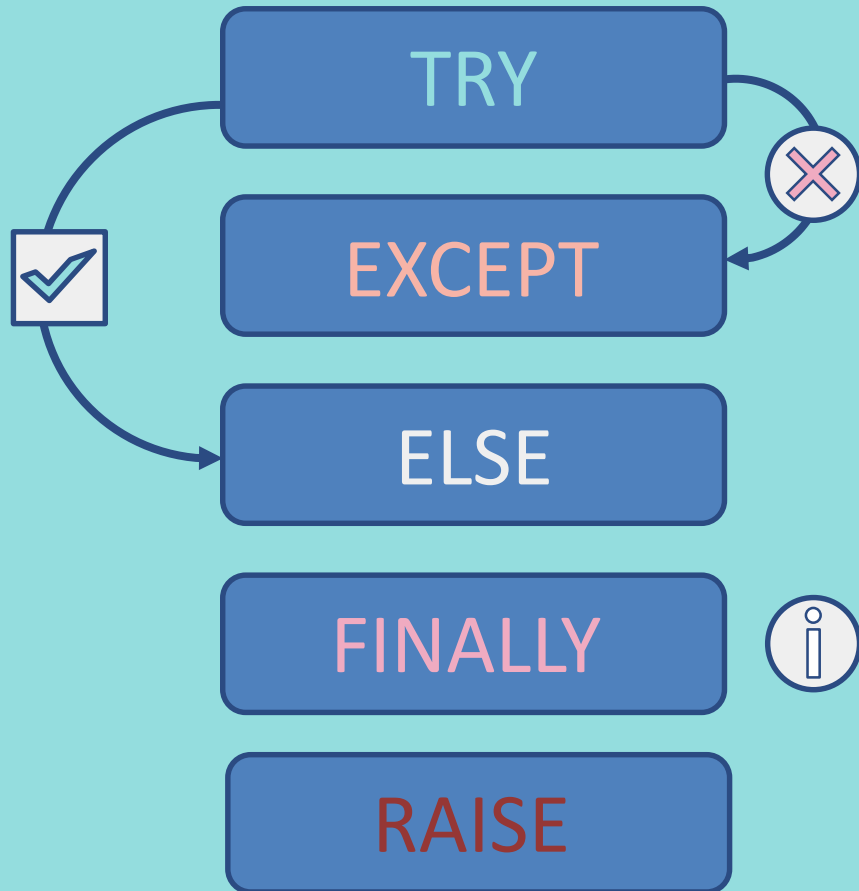


ВОПРОСЫ

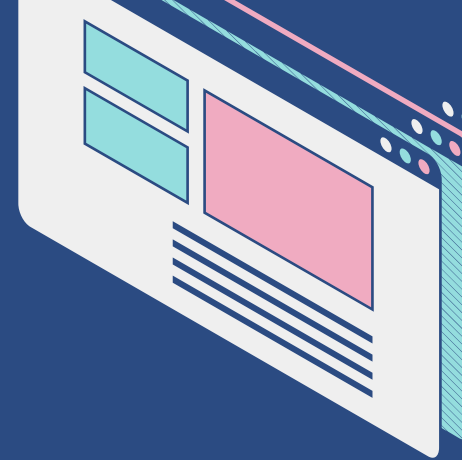
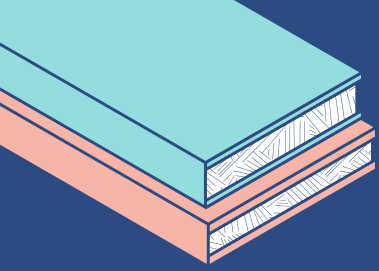
КЛЮЧЕВЫЕ ЗНАНИЯ

- Принципы работы исключений
- Обработка исключений.
- Пользовательские исключения

Что такое исключения?



Исключения – это события, возникающие во время выполнения программы, которые могут привести к ошибкам или аномальному завершению программы. Они обычно возникают из-за некорректного кода или непредвиденных ситуаций, таких как деление на ноль, доступ к несуществующему файлу или использование неопределенной переменной.



Зачем исключения в Python?

Представьте, что ваша программа пытается открыть файл, которого не существует, или делит число на ноль. В подобных ситуациях возникает ошибка, которая может привести к аварийному завершению программы. Однако, с помощью исключений, мы можем предвидеть такие ошибки и корректно их обрабатывать, обеспечивая стабильную работу программы.

Основные цели использования исключений включают:

- Повышение надежности программы за счет обработки ошибок.
- Упрощение отладки и поиска проблем.
- Сокращение количества ошибок, возникающих в уже работающей программе.

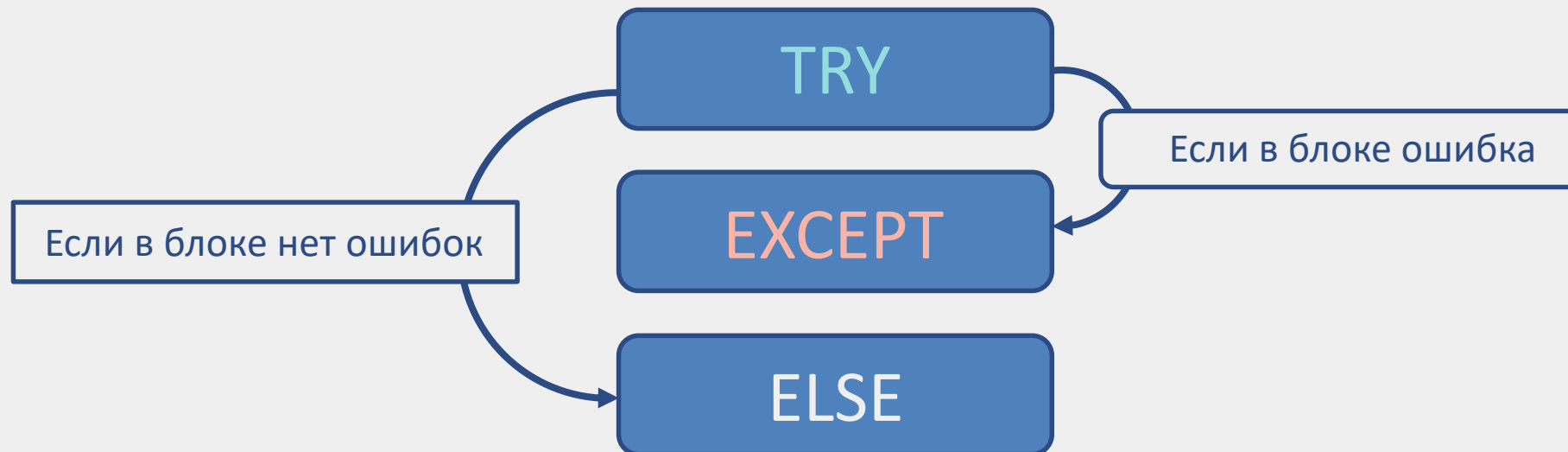


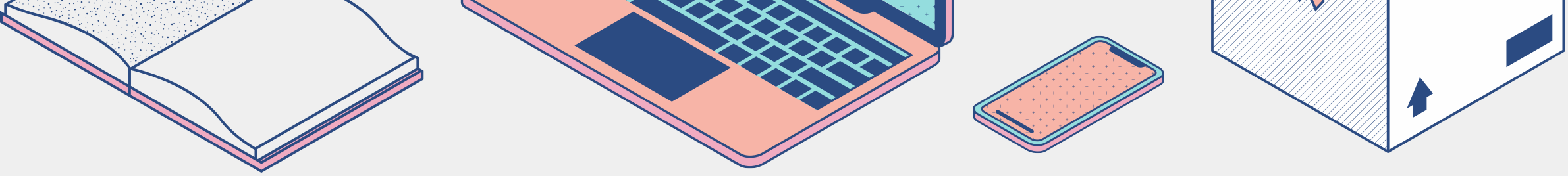


Блоки исключений: TRY

- С помощью блока **try** Python проверяет код на наличие исключений. Если в **try** встречается ошибка, выполнение переходит к первому блоку **except**.

```
1 try:
2     print(undefined_variable)
3 except NameError:
4     print('Переменная не определена.')
✓ 0.0s
Переменная не определена.
```





Блоки исключений: EXCEPT

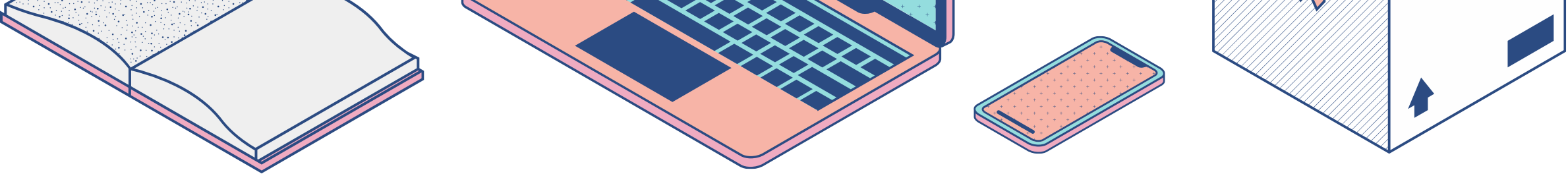
- С помощью блоков **except** можно обрабатывать несколько исключений разными способами. Каждый **except** соответствует определённому типу ошибки.
- Блоков **except** может быть столько, сколько необходимо.

```
1 try:
2     n = '2' + 2
3 except TypeError:
4     print('Сообщение о TypeError')
5 except ZeroDivisionError:
6     print('Сообщение о ZeroDivisionError')
```

✓ 0.0s

Сообщение о TypeError





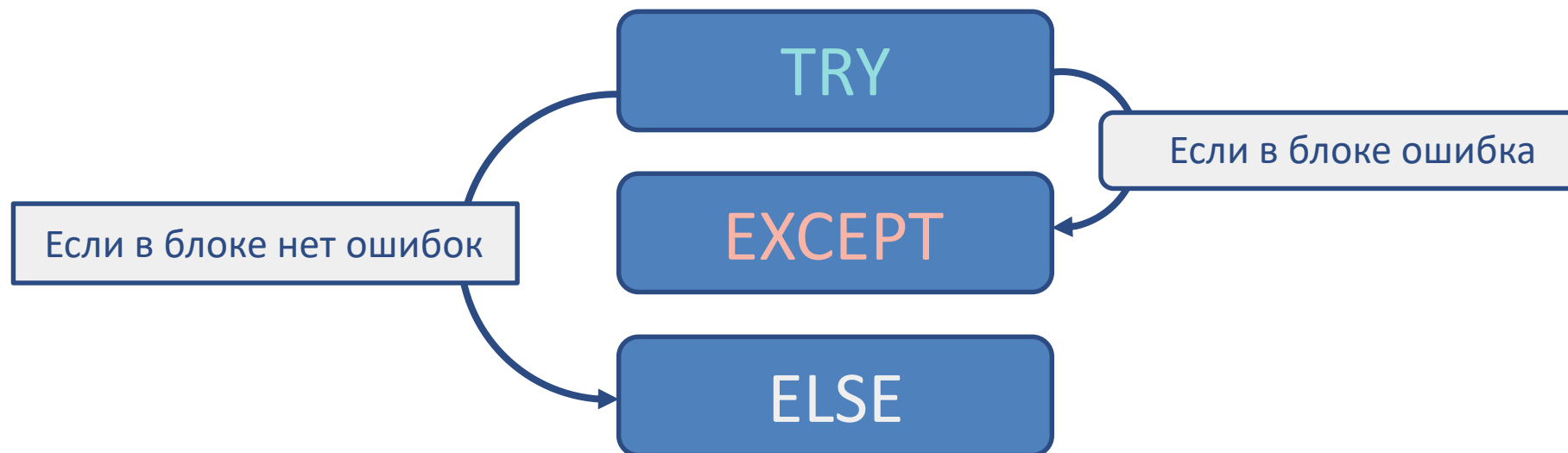
Блоки исключений: ELSE

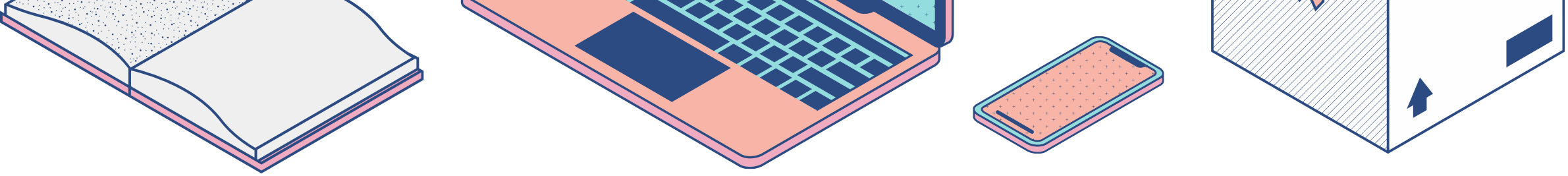
- Код в **else** выполняется, если **try** не нашёл исключений.

```
1 try:
2     n = int(input('Введите число: '))
3 except ValueError:
4     print('Это не число!')
5 else:
6     print(f'Вы ввели число {n}')
```

✓ 1.4s

Вы ввели число 123





Блоки исключений: FINALLY

- В **finally** помещают код, который будет выполняться независимо от того, была ли найдена ошибка или нет. Часто этот блок используют для работы с файлами, чтобы закрыть документ.

```
1 try:
2     f = open('my_file.txt')
3     # Действия с файлом
4 except FileNotFoundError:
5     print('Файл не найден.')
6 finally:
7     f.close()
```

0.0s

Файл не найден.

NameError Traceback (most recent call last)

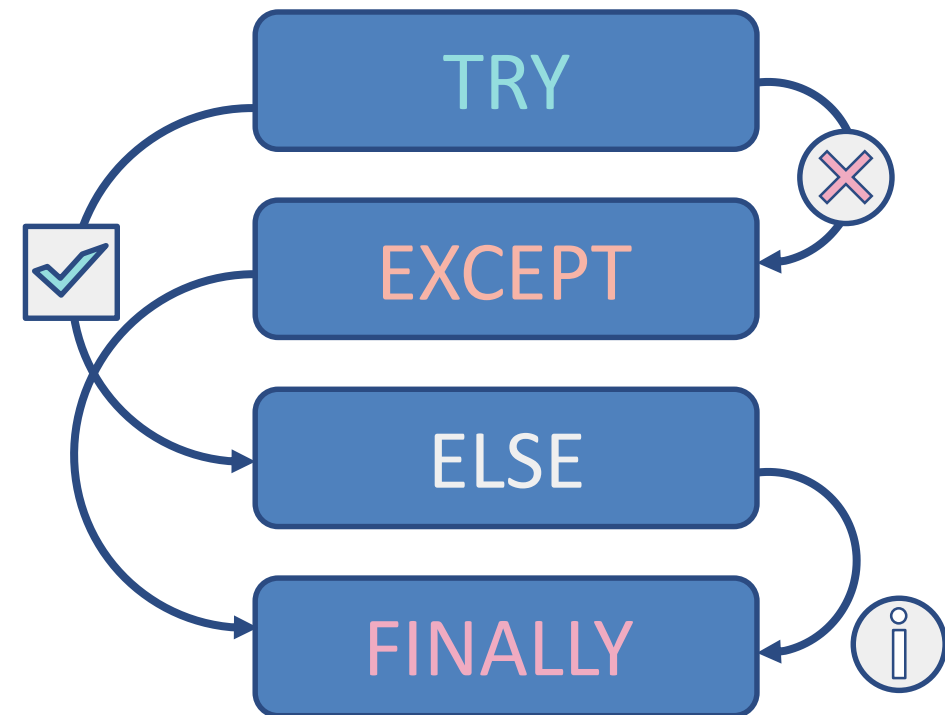
Cell In[5], line 7

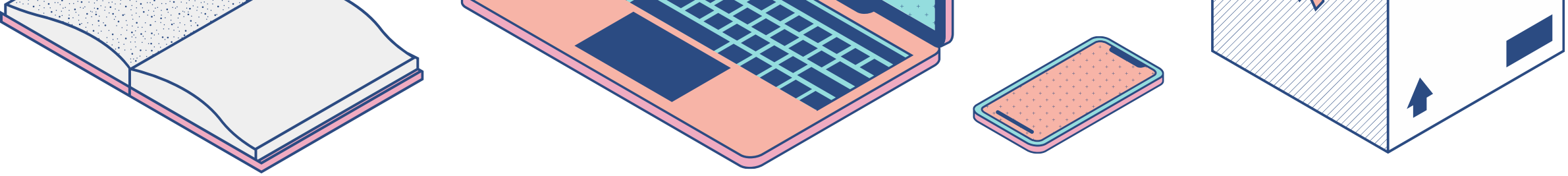
5 print('Файл не найден.')

6 finally:

----> 7 f.close()

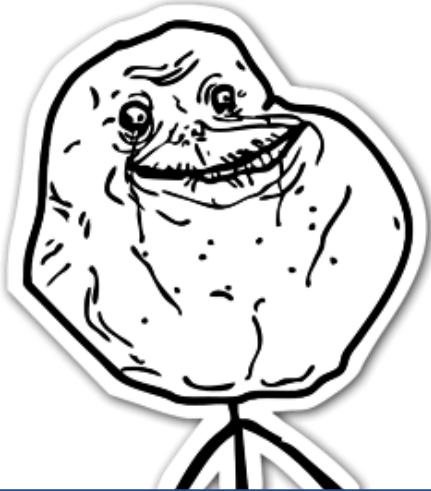
NameError: name 'f' is not defined





Блоки исключений: RAISE

- Команда **raise** в Python используется для принудительного вызова исключения. Это может быть полезно, если мы столкнулись с условием, которое должно остановить выполнение программы или вызвать ошибку.



RAISE

```
1 raise ValueError('Недопустимое значение')
ⓧ ✨ 0.0s

-----
ValueError                                Traceback (most recent call last)
Cell In[8], line 1
----> 1 raise ValueError('Недопустимое значение')

ValueError: Недопустимое значение
```

ОБРАБОТКА МНОЖЕСТВЕННЫХ ИСКЛЮЧЕНИЙ

С помощью блоков `except` можно обрабатывать несколько исключений разными способами. Каждый `except` соответствует определённому типу ошибки.

Например, если в блоке `try` происходит исключение `TypeError`, будет выполнен первый блок `except`, и аналогично для `ZeroDivisionError`.

Иногда возникает необходимость обрабатывать несколько типов исключений одинаковым образом. В этом случае можно перечислить эти исключения в одном блоке `except`, разделив их запятыми:

```
1 try:
2     n = '2' + 2
3 except TypeError:
4     print('Сообщение о TypeError')
5 except ZeroDivisionError:
6     print('Сообщение о ZeroDivisionError')
```

✓ 0.0s

Сообщение о TypeError

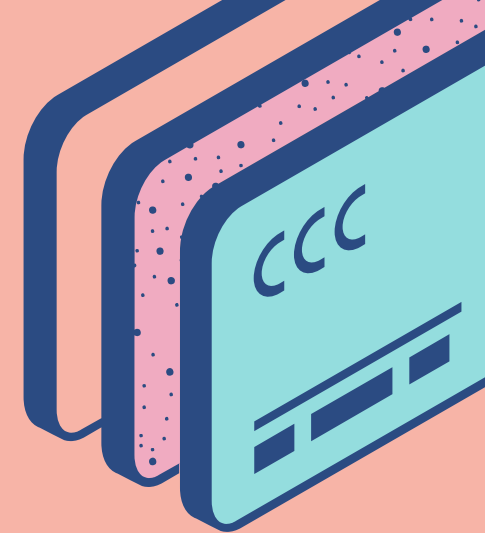
```
1 try:
2     n = '1' + 1 # Код, который может вызвать исключение TypeError или ZeroDivisionError
3 except (TypeError, ZeroDivisionError):
4     print('Обнаружена ошибка TypeError или ZeroDivisionError')
```

✓ 0.0s

Обнаружена ошибка TypeError или ZeroDivisionError

Пользовательские исключения

- Если нужен свой собственный обработчик — пишутся пользовательские исключения
- Для создания собственного исключения достаточно определить новый класс, который наследуется от базового класса Exception или от любого другого встроенного исключения
- Применяется как исключения «из коробки»



```
1 class ValidationError(Exception):  
2     pass
```

```
1 def person_age(age):  
2     if age < 0:  
3         raise ValidationError('Возраст не может быть отрицательным')  
4     elif age > 120:  
5         raise ValidationError('Возраст не может быть больше 120')  
6     return True  
7  
8 try:  
9     person_age(150)  
10 except ValidationError as e:  
11     print(e)
```

✓ 0.0s

Возраст не может быть больше 120

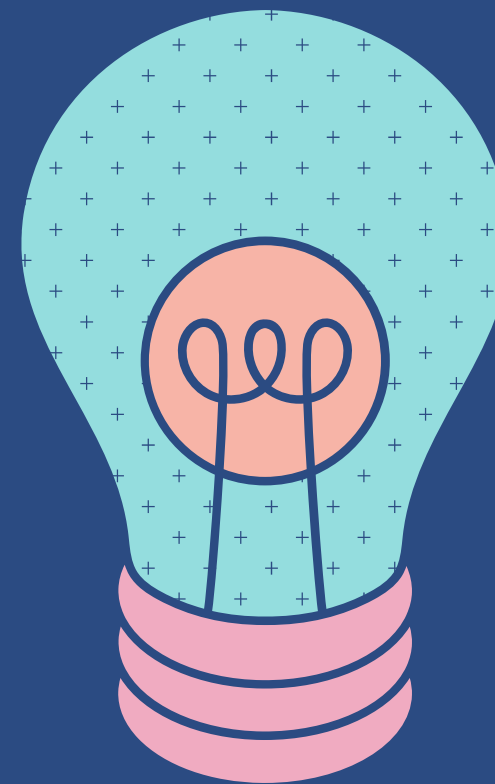


Следующие шаги нашего обучения

- Параллельное программирование
- Паттерны проектирования
- Тестирование

«Информация в чистом виде — это не
знание. Настоящий источник знания —
это опыт.»

АЛЬБЕРТ ЭЙНШТЕЙН



У вас есть какие-то вопросы?

Оставьте их себе! Надеюсь, что вы узнали
что-то новое.

Позвоните мне

+375 (33) 398-54-37

.....

Электронная почта

kordebalet67@gmail.com

.....

Telegram

@Kordebalet





КАФЕДРА №13

Назначение и область применения исключений

ЛИТВИНОВ ВЛАДИСЛАВ